MASTER BUILD PROMPT

SMARTBANK AI — AI-POWERED SMART BANKING & FINANCIAL MANAGEMENT SYSTEM

Domain

AI + FinTech + Cybersecurity

Core Focus

Smart Banking + Personal Financial Management + Phishing Detection + Malware Detection + Ransomware Detection + Explainable AI

Primary Technology Requirement

Core Java

Hackathon Objective

Build a **functional, deployable, professional-quality Smart Banking and Financial Management web application** that combines financial intelligence with an integrated cybersecurity defense layer.

This must NOT be a simple CRUD banking application and must NOT be only a UI mockup.

The final system must demonstrate:

> **Detect → Analyze → Explain → Alert → Respond → Record**

The project should be impressive enough for a hackathon jury while remaining technically realistic and understandable to a CSE student.

---

1. PROJECT IDENTITY

Project Name

SmartBank AI

Tagline

"Bank Smarter. Detect Threats. Protect Every Transaction."

One-Line Pitch

SmartBank AI is an intelligent Core Java banking platform that manages personal finances while continuously detecting suspicious financial activity, phishing, malware and ransomware-like threats through an explainable AI-driven security engine.

---

2. PRIMARY GOALS

The system must:

1. Provide secure user authentication.
2. Manage simulated bank accounts.
3. Support simulated transactions and transfers.
4. Track income and expenses.
5. Automatically categorize spending.
6. Provide budgeting and financial goals.
7. Generate spending insights.
8. Predict upcoming financial expenses.
9. Detect suspicious transactions.
10. Detect phishing URLs/messages.
11. Detect suspicious uploaded files without executing them.
12. Detect ransomware-like behavior safely through simulation.
13. Correlate multiple security events.
14. Generate explainable risk scores.
15. Provide an integrated Security Intelligence Center.
16. Provide an Admin/Security Analyst dashboard.
17. Maintain audit logs.
18. Generate financial and security reports.
19. Provide a safe Cyber Threat Simulation Lab.
20. Provide a clear hackathon demonstration workflow.

---

3. NON-NEGOTIABLE TECHNOLOGY RULE

The application backend and core business logic MUST use:

* Java
* Core Java
* Object-Oriented Programming
* Collections Framework
* Exception Handling
* Generics
* Interfaces
* JDBC

* Java I/O
* Java concurrency
* Java Security APIs where appropriate

Do NOT implement the core business logic using:

* Python
* Node.js
* PHP
* Django
* Flask
* Spring Boot

If a web framework is required, use lightweight Java web technologies.

Preferred stack:

HTML + CSS + JavaScript → Java Servlet/JSP → Service Layer → AI/Risk Engine → JDBC → Database

Use vanilla JavaScript for frontend interaction.

---

4. DATABASE

Use a relational database.

Preferred:

MySQL

If external database configuration would prevent easy hackathon execution, provide an embedded development option such as H2/SQLite while keeping the DAO layer database-independent.

Create tables for:

* users
* accounts
* transactions
* beneficiaries
* login_history
* budgets
* financial_goals
* recurring_payments
* fraud_alerts

* phishing_alerts
* malware_alerts
* ransomware_alerts
* security_events
* incidents
* notifications
* audit_logs
* support_tickets

Use:

* Primary keys
* Foreign keys
* Constraints
* Indexes
* PreparedStatement
* Proper normalization

NEVER construct SQL using unsafe string concatenation.

---

5. USER ROLES

Implement RBAC.

CUSTOMER

Customer can:

* Register
* Login
* Logout
* View account
* View balance
* View transactions
* Perform simulated transfers
* Add beneficiaries
* Manage budgets
* Create financial goals
* View spending analytics
* Use AI Advisor
* Scan suspicious URLs/messages
* Submit safe files for security analysis
* View security alerts
* View reports
* Manage profile

ADMIN

Admin can:

* View users
* Monitor transactions
* Monitor security events
* Review fraud alerts
* Review phishing alerts
* Review malware alerts
* Review ransomware alerts
* View incidents
* Review risk scores
* Suspend simulated accounts
* Manage alerts
* Generate reports
* View system analytics

SECURITY ANALYST

Security Analyst can:

* Investigate incidents
* Examine security events
* View incident timelines
* Review device/IP information
* Analyze phishing indicators
* Analyze malware indicators
* Analyze ransomware indicators
* Mark alerts as confirmed/false positive
* Add investigation notes
* Review recommended responses

---

6. AUTHENTICATION & SECURITY

Implement:

* Registration
* Login
* Logout
* Password hashing
* Session management
* Role-based authorization
* Failed-login detection

* Account lockout simulation
* Login history
* Suspicious login detection
* Device/IP information where appropriate
* OTP/MFA simulation
* Input validation

Never store passwords as plaintext.

Never expose stack traces to users.

Never hard-code real credentials.

---

7. SMART BANKING MODULE

Implement:

Accounts

Support:

* Savings
* Current
* Fixed Deposit

Display:

* Account number
* Account type
* Balance
* Status
* Recent activity

Transactions

Support simulated:

* Deposit
* Withdrawal
* Transfer
* Payment

Include:

* Amount

* Date/time
* Merchant
* Category
* Transaction type
* Status
* Risk score

Use Indian Rupee formatting.

---

8. AI FINANCIAL MANAGEMENT

Create:

SmartBank Advisor

The advisor analyzes transaction history and provides understandable financial insights.

Examples:

* Spending increased in a particular category.
* Upcoming recurring payments may affect the budget.
* Current spending may exceed the monthly budget.
* A transaction differs significantly from normal behavior.
* Savings progress is below the selected goal.

Every recommendation must explain the reasoning.

Clearly label predictions and recommendations as **estimates**, not guaranteed financial advice.

---

9. BUDGET MANAGEMENT

Allow users to create:

* Monthly budget
* Food budget
* Travel budget
* Shopping budget
* Education budget
* Entertainment budget
* Other categories

Show:

* Budget limit
* Current spending
* Remaining amount
* Percentage used
* Warning threshold

Example:

Shopping Budget: ₹10,000

Used: ₹8,700

Usage: 87%

Generate an alert when the user approaches the limit.

---

10. FINANCIAL GOALS

Users can create goals such as:

* Laptop
* Education
* Emergency fund
* Travel
* Savings

Display:

* Target amount
* Current amount
* Remaining amount
* Target date
* Progress percentage
* Suggested monthly contribution

---

11. SPENDING ANALYTICS

Create interactive charts for:

* Daily spending

* Weekly spending
* Monthly spending
* Category spending
* Income vs expenses
* Savings trend
* Recurring expenses
* Largest transactions

Example:

```text
Monthly Income      ₹50,000
Monthly Expenses    ₹31,500
Savings             ₹18,500
Savings Rate        37%
```

---

12. FINANCIAL PREDICTION

Implement a lightweight Java prediction engine.

Use deterministic techniques such as:

* Moving average
* Historical trend
* Recurring-payment analysis
* Budget projection

Predict:

* Next month's estimated expenses
* Upcoming recurring payments
* Possible budget overruns
* Savings trajectory

Clearly label outputs as:

Estimated / Predicted

Do not falsely claim that a trained ML model is being used if it is not.

---

13. TRANSACTION FRAUD DETECTION

Create:

Fraud Detection Engine

Analyze:

* Transaction amount
* Transaction frequency
* Transaction time
* Device
* Previous transaction behavior
* New beneficiary
* Merchant
* Unusual spending pattern
* Rapid transaction sequence
* Historical baseline

Generate:

Risk Score

Example:

87/100

Risk Level

* LOW
* MEDIUM
* HIGH
* CRITICAL

Explanation

Example:

> High risk because the transaction is significantly different from the customer's normal behavior and involves a newly added beneficiary.

---

14. EXPLAINABLE AI ENGINE

Create one reusable scoring engine shared by all security modules.

Architecture:

```text
Input Event
    ↓
Feature Extraction
    ↓
Signal Detection
    ↓
Weighted Risk Engine
    ↓
Risk Score
    ↓
Risk Classification
    ↓
Explanation Generator
    ↓
Recommended Action
```

Example formula:

```text
Risk Score =
Σ(signal weight × signal triggered)
```

Normalize to:

0–100

Every AI/security decision MUST explain:

1. What happened?
2. Why was it detected?
3. What evidence was considered?
4. What is the risk?
5. What should the user/admin do?

Do not call a rule engine a trained AI model.

If no real ML model is connected, label it:

Explainable AI-assisted rule-based engine

---

15. PHISHING DETECTION

Create:

AI Phishing Shield

Users can submit:

* URL
* SMS
* Email/message text

Analyze:

* Suspicious domain
* Lookalike domain
* Domain mismatch
* Suspicious TLD
* IP-based URL
* Excessive subdomains
* Urgency language
* Credential requests
* Fake login indicators
* Suspicious redirects
* Impersonation indicators

Generate:

Phishing Risk Score: 92/100

Classification:

* SAFE
* SUSPICIOUS
* HIGH RISK

Explain exactly which signals caused the score.

Do NOT automatically visit or execute suspicious URLs.

Use safe simulated analysis.

---

16. MALWARE DETECTION

Create:

AI Malware Guard

Never execute uploaded files.

Use defensive analysis only.

Analyze:

* File extension
* File type
* Magic bytes
* File size
* Hash
* Filename patterns
* Double extensions
* Safe test signatures
* Simulated behavioral indicators

Example:

```text
statement.pdf.exe
```

Possible result:

```text
Risk Score: 88/100
Risk Level: HIGH

Reasons:
• Double extension
• File-type mismatch
• Suspicious executable indicator
```

Flagged files must be quarantined rather than executed.

---

17. RANSOMWARE DETECTION

Create:

Ransomware Sentinel

Because this is a banking application, implement ransomware detection as a **safe application/data-layer simulation**.

Monitor simulated indicators such as:

* Rapid file modifications
* Rapid record modifications
* Abnormal rename activity
* Suspicious extension changes
* Abnormal access bursts
* Decoy/honeypot access

Create safe decoy records/files.

When the simulation detects ransomware-like activity:

1. Generate alert.
2. Calculate risk score.
3. Identify affected asset.
4. Create incident timeline.
5. Recommend defensive action.
6. Notify admin/security analyst.
7. Record the incident.

Do NOT create actual ransomware.

Do NOT encrypt user files.

---

18. ALWAYS-ON SECURITY MONITOR

Use Java concurrency such as:

```text
ScheduledExecutorService
```

The monitor should observe application security events such as:

* Failed logins
* Suspicious login
* Transaction velocity
* Transaction anomalies
* File uploads
* Phishing submissions

* Honeypot events

Flow:

```text
MONITORING
   ↓
THREAT DETECTED
   ↓
ANALYZING
   ↓
RISK CALCULATION
   ↓
RESPONSE
   ↓
INCIDENT LOGGED
   ↓
MONITORING
```

The monitoring process must not block normal web requests.

---

19. INCIDENT CORRELATION

This is a major hackathon differentiator.

Do not display every alert as an unrelated event.

Correlate related events.

Example:

```text
Suspicious Login
      ↓
New Device
      ↓
Password Reset
      ↓
New Beneficiary
      ↓
Large Transaction
      ↓
Possible Account Takeover
```

Generate one unified incident containing:

* Incident ID
* Threat type
* Timeline
* Related events
* Risk score
* Evidence
* Affected account
* Severity
* Recommended action
* Status
* Analyst notes

---

20. SECURITY INTELLIGENCE CENTER

Create the main cybersecurity dashboard.

Display:

Overall Security Score

Example:

94/100

Threat Status

```text
Fraud          LOW
Phishing       MEDIUM
Malware        LOW
Ransomware     LOW
Account Risk   LOW
```

Show:

* Live monitoring status
* Threat statistics
* Incident timeline
* Recent alerts
* Risk scores
* Threat categories

* Security trends
* Recommended actions

---

21. CYBER THREAT SIMULATION LAB

Create a dedicated demonstration page.

Buttons:

```text
[Simulate Fraud]

[Simulate Phishing]

[Simulate Malware]

[Simulate Ransomware]

[Simulate Account Takeover]
```

These must generate harmless synthetic events.

The judge should be able to click a button and immediately see:

```text
EVENT DETECTED
    ↓
ANALYZING
    ↓
RISK SCORE
    ↓
EXPLANATION
    ↓
ALERT
    ↓
RECOMMENDED RESPONSE
    ↓
INCIDENT REPORT
```

This should be one of the most visually impressive parts of the application.

---

22. AUTOMATED/GUIDED RESPONSE

Use safe responses.

Examples:

Suspicious Login

Recommend:

Require MFA verification

Suspicious Transaction

Recommend:

Temporarily restrict transaction and verify

Phishing

Recommend:

Block suspicious action

Malware

Recommend:

Quarantine file

Ransomware-like event

Recommend:

Freeze affected simulated account/session and investigate

Critical actions should use human confirmation where appropriate.

Principle:

> **Detect → Explain → Alert → Recommend → Confirm → Record**

---

23. DASHBOARD

Create a professional banking dashboard.

Top navigation:

* Dashboard
* Accounts
* Transactions
* Finance
* AI Advisor
* Security Center
* Threat Lab
* Alerts
* Reports
* Profile

Dashboard cards:

* Total Balance
* Monthly Income
* Monthly Expenses
* Savings
* Financial Health
* Security Score
* Fraud Risk

Main sections:

1. Spending Overview
2. Recent Transactions
3. AI Insights
4. Security Alerts
5. Financial Goals
6. Upcoming Payments
7. Live Security Status

---

24. ADMIN DASHBOARD

Display:

* Total users
* Active accounts
* Transactions today
* Suspicious transactions
* Fraud alerts

* Phishing alerts
* Malware alerts
* Ransomware alerts
* Active incidents

Charts:

* Threats over time
* Threat categories
* Risk levels
* Transaction anomalies
* Incident severity

---

25. AUDIT LOGGING

Maintain an application audit trail.

Record:

* User ID
* Event type
* Timestamp
* Action
* Result
* Risk level
* Incident ID where applicable
* Device/IP information where appropriate

Example events:

```text
LOGIN_SUCCESS
LOGIN_FAILURE
TRANSACTION_CREATED
BENEFICIARY_ADDED
PHISHING_SCAN
MALWARE_SCAN
FRAUD_ALERT
RANSOMWARE_ALERT
ADMIN_REVIEW
SECURITY_RESPONSE
```

Security events must persist in the database.

---

26. REPORT GENERATION

Generate:

Financial Report

Include:

* Income
* Expenses
* Savings
* Categories
* Goals
* Recurring payments

Security Report

Include:

* Fraud events
* Phishing detections
* Malware detections
* Ransomware alerts
* Suspicious logins
* Incident timeline
* Risk scores
* Actions taken

Make reports printable/downloadable where practical.

---

27. JAVA PROJECT ARCHITECTURE

Use clean layered architecture.

```text
SmartBankAI/
|
 ├──── src/
|    └──── main/
|         └──── java/
|              └──── com/smartbank/
|
```

```
|               ├──── model/
|               ├──── dao/
|               ├──── service/
|               ├──── controller/
|               ├──── ai/
|               ├──── security/
|               |    ├──── phishing/
|               |    ├──── malware/
|               |    ├──── ransomware/
|               |    └──── monitor/
|               ├──── banking/
|               ├──── finance/
|               ├──── auth/
|               └──── util/
|
 ├──── src/main/webapp/
|    ├──── WEB-INF/
|    ├──── css/
|    ├──── js/
|    ├──── assets/
|    └──── *.jsp
|
 └──── database/
     └──── schema.sql
```

Separate:

Presentation

↓

Controller

↓

Service

↓

AI/Security Engine

↓

DAO

↓

Database

---

28. OBJECT-ORIENTED DESIGN

Demonstrate:

* Encapsulation
* Inheritance
* Polymorphism
* Abstraction
* Interfaces
* Generics
* Collections
* Enums
* Exception handling
* Multithreading

Create a common interface:

```java
interface ThreatDetector {
    ThreatResult analyze(SecurityEvent event);
}
```

Implement:

```text
FraudDetector
PhishingDetector
MalwareDetector
RansomwareDetector
```

This makes the system modular and extensible.

---

29. CUSTOM EXCEPTIONS

Create appropriate exceptions such as:

```text
InvalidLoginException
```

UnauthorizedAccessException
InsufficientBalanceException
InvalidTransactionException
SuspiciousTransactionException
SecurityScanException
DatabaseException
```

Never expose technical stack traces to users.

---

# 30. UI/UX REQUIREMENTS

The website must look like a modern FinTech application.

Use:

* Responsive design
* Sidebar
* Cards
* Charts
* Tables
* Risk badges
* Notifications
* Timelines
* Progress bars
* Search
* Filters
* Modal dialogs
* Status indicators

Support:

**Light Mode + Dark Mode**

The UI must work well on:

* Laptop
* Projector
* Mobile-sized viewport

Do NOT make it look like a basic college CRUD project.

---

# 31. HACKATHON DIFFERENTIATORS
```

The final application must emphasize these five differentiators:

1. Finance + Cybersecurity

One platform combines personal financial management and cyber threat defense.

2. Explainable AI

Instead of simply saying:

"Fraud detected."

Show:

"Fraud detected because of unusual amount + new beneficiary + unusual time + unfamiliar device."

3. Cross-Event Correlation

Convert individual alerts into meaningful incidents.

4. Safe Cyber Threat Simulation

Demonstrate phishing, malware and ransomware detection without executing malicious code.

5. Real-Time Security Monitoring

Use a Java background monitoring engine to demonstrate continuous security awareness.

---

32. SAMPLE DEMO DATA

Create fictional demo data.

Example:

```text
Demo Customer

Balance: ₹75,000
Monthly Income: ₹50,000
Monthly Expenses: ₹31,500
```

Savings: ₹18,500
Security Score: 94/100
```

Create scenarios for:

1. Normal transaction
2. Suspicious transaction
3. Safe URL
4. Phishing URL
5. Safe file
6. Suspicious file
7. Ransomware-like event
8. Suspicious login
9. Account takeover sequence

Never use real financial information.

---

# 33. PRIMARY HACKATHON DEMO FLOW

The entire demo should fit into approximately 5–7 minutes.

### STEP 1 — Login

Login as demo customer.

### STEP 2 — Dashboard

Show:

* Balance
* Expenses
* Savings
* Financial insights
* Security score

### STEP 3 — AI Finance

Open SmartBank Advisor.

Show an explainable spending insight.

### STEP 4 — Suspicious Transaction

Trigger a suspicious transaction.
```

Show:

```text
Risk Score: 87/100
HIGH RISK

Reasons:
• New beneficiary
• Unusual amount
• Unusual time
• New device
```

STEP 5 — Phishing

Paste a safe demonstration phishing URL.

Show:

```text
PHISHING DETECTED

Risk: 92/100

Reasons:
• Lookalike domain
• Credential request
• Urgency language
```

STEP 6 — Malware

Upload the harmless demo file:

```text
statement.pdf.exe
```

Show:

```text
HIGH RISK

Double extension detected
File type mismatch
File quarantined
```

```

### STEP 7 — Ransomware

Trigger the safe honeypot simulation.

Show:

```text
RANSOMWARE-LIKE ACTIVITY

Threat Detected
      ↓
Analyzing
      ↓
Risk: CRITICAL
      ↓
Session/Account Restricted
      ↓
Incident Created
```

STEP 8 — Incident Correlation

Show all related events in a timeline.

STEP 9 — Admin

Switch to Admin/Security Analyst.

Show:

* Threat dashboard
* Incident
* Risk score
* Evidence
* Recommended response
* Audit trail

STEP 10 — Final Impact Statement

End the demonstration by showing that SmartBank AI does not merely detect threats.

It:

Detects → Explains → Correlates → Responds → Records
```

---

34. SECURITY RULES

NEVER:

* Execute uploaded malware.
* Encrypt real files.
* Deploy ransomware.
* Use real banking credentials.
* Use real customer data.
* Store plaintext passwords.
* Hard-code credentials.
* Claim 100% fraud detection.
* Claim a rule engine is a trained ML model.
* Perform destructive actions automatically.

All cybersecurity demonstrations must remain safe and simulated.

---

35. PERFORMANCE

Ensure:

* Efficient Java collections
* Efficient database queries
* Pagination
* Background monitoring
* Non-blocking security monitoring
* Responsive UI
* Minimal unnecessary database calls

---

36. DOCUMENTATION

Generate a professional README containing:

* Project overview
* Problem statement
* Solution
* Innovation
* Features
* Architecture
* Technology stack

* Database design
* Security model
* AI/risk engine
* Installation
* Run instructions
* Demo credentials
* Demo scenario
* Limitations
* Future improvements

Also generate:

* Architecture diagram
* Database ER diagram
* Data flow diagram
* Threat detection flow
* Demo guide

---

37. IMPLEMENTATION STRATEGY

Do NOT attempt to create everything as disconnected mockups.

Implement in this order:

Phase 1

Project setup + database + authentication

Phase 2

Accounts + transactions + beneficiaries

Phase 3

Budgeting + financial goals + analytics

Phase 4

AI Advisor + financial prediction

Phase 5

Fraud detection

Phase 6

Phishing detection

Phase 7

Malware detection

Phase 8

Ransomware simulation + honeypot

Phase 9

Always-on monitoring

Phase 10

Incident correlation

Phase 11

Security Intelligence Center

Phase 12

Admin dashboard + reports

Phase 13

Hackathon demo simulation

Phase 14

Testing + UI polish + documentation

Each phase must produce a working feature before moving to the next.

---

38. ACCEPTANCE CRITERIA

Before declaring the project complete, verify:

```text
[ ] Application starts successfully
[ ] Registration works
[ ] Login works
```

[ ] Logout works
[ ] RBAC works
[ ] Accounts work
[ ] Transactions work
[ ] Transfers work
[ ] Budgets work
[ ] Financial goals work
[ ] Analytics work
[ ] AI Advisor works
[ ] Financial prediction works
[ ] Fraud detection works
[ ] Phishing detection works
[ ] Malware scanning works safely
[ ] Ransomware simulation works
[ ] Background monitoring works
[ ] Incident correlation works
[ ] Security Center works
[ ] Admin dashboard works
[ ] Audit logging works
[ ] Reports work
[ ] Threat Simulation Lab works
[ ] Risk explanations work
[ ] Input validation works
[ ] Error handling works
[ ] Database persistence works
[ ] Responsive UI works
[ ] Demo data works
[ ] README works
```

---

# 39. FINAL ANTIGRAVITY BUILD DIRECTIVE

Build a **functional end-to-end application**, not a visual mockup.

The final system must contain:

**Frontend**

*

**Core Java Web Backend**

*

**Database**
```

*

Banking Module

*

Financial Management

*

Explainable AI/Risk Engine

*

Fraud Detection

*

Phishing Detection

*

Malware Detection

*

Ransomware Simulation

*

Real-Time Monitoring

*

Incident Correlation

*

Admin/Security Dashboard

*

Audit Logging

*

Threat Simulation Lab

*

Reports

The application must be:

Functional + Secure + Explainable + Demonstrable + Modular + Hackathon-ready.

If a genuine AI/ML model is unavailable, use a transparent Java rule-weighted engine and clearly label it as an **AI-assisted explainable prototype**.

Do not invent AI capabilities that are not actually implemented.

The central product philosophy is:

> **"SmartBank AI continuously understands financial behavior, detects cyber threats, explains why risks occur, guides safe responses, and helps users make informed financial decisions."**

Build the project as a **professional AI + FinTech + Cybersecurity solution suitable for a national-level hackathon/Buildathon demonstration.**